

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi – 590018



AIML Faculty Load Allocator

Submitted in partial fulfillment of the requirements as a part of the
curriculum,

**Bachelors of Engineering in Artificial Intelligence and Machine
Learning**

Submitted by

Sukanya Kalasad 1CR24AI130

Sparsha S R 1CR24AI128

Shweta Pawar 1CR24AI126

Under the Guidance of

Dr. Shyam P Joy

Head Of Department

Department of Artificial Intelligence and Machine Learning



Department of Artificial Intelligence and Machine Learning

CMR INSTITUTE OF TECHNOLOGY

132, AECS Layout, Kundalahalli, ITPL Main Rd, Bengaluru – 560037



CERTIFICATE

Certified that the Mini Project entitled “**AIML Faculty Load Allocator**” is presented by

Sukanya Kalasad 1CR24AI130

Sparsha S R 1CR24AI128

Shweta Pawar 1CR24AI126

bonafide student(s) of **CMR Institute of Technology** in partial fulfillment of the requirements as a part of the curriculum, **Bachelors of Engineering in Artificial Intelligence and Machine Learning**, of **Visvesvaraya Technological University, Belagavi** during the year **2025-26**. It is certified that all correction/suggestion indicated for Internal Assessment have been incorporated in the report deposited in the departmental library. The Entrepreneurial Project report has been approved as it satisfies the academic requirements in respect of the Technical Seminar prescribed for the bachelor of engineering degree.

(Dr. Shyam P Joy)

(Dr. Shyam P Joy)

Signature of the Guide

Signature of the HOD

DECLARATION

We, the undersigned, students of **CMR Institute of Technology**, hereby declare that this mini project report titled

“AIML Faculty Load Allocator”

is a record of our original work done under the supervision of [Shyam P Joy], [HOD], Department of [AIML]. And has not formed the basis of award of any other degree, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

1. Sukanya Kalasad (USN: 1CR24AI130)
2. Sparsha S R (USN: 1CR24AI128)
3. Shweta Pawar (USN: 1CR24AI126)

ABSTRACT

The AIML Faculty Load Allocator is a web-based application developed to automate and simplify the process of faculty workload allocation in academic institutions. In many colleges and universities, faculty workload management is still performed manually using spreadsheets and traditional record-keeping methods. This manual approach is time-consuming, error-prone, and difficult to maintain when handling a large number of faculty members, subjects, laboratory sessions, and administrative responsibilities. Inconsistent faculty naming conventions and repetitive calculations further increase the complexity of workload preparation and report generation.

The proposed system addresses these challenges by providing an automated platform for processing faculty allocation data through uploaded Excel sheets. The application automatically calculates theory workload, laboratory workload, and TRCPIE allocation while generating structured reports for academic departments. The system also implements faculty name matching and validation techniques to reduce duplication errors and improve data consistency.

The application is developed using React, Vite, and TypeScript for the frontend, while FastAPI is used for backend API development and request handling. Pandas and OpenPyXL libraries are utilized for Excel file processing, data manipulation, and automated report generation. The system was tested using different departmental datasets, and the results demonstrated accurate workload calculation, faster processing time, and improved efficiency compared to manual methods. The project successfully reduces administrative effort, minimizes human errors, and provides a reliable and user-friendly solution for faculty workload management in educational institutions.

ACKNOWLEDGEMENTS

It is my proud privilege and duty to acknowledge the kind of help and guidance received from several people in preparation of this report. Apart from my own, the success of this report depends largely on the encouragement and guidelines of many others. It would have not been possible to prepare this report in this form without their valuable help, co-operation and guidance.

I would like to express my deep sense of gratitude to our Principal **Dr. Sanjay Jain**, CMR Institute of Technology, Bangalore for his motivation and for creating an inspiring atmosphere in college by providing state of the art facilities for preparation and delivery of this report.

My sincere thanks to **Dr. Shyam P Joy** , Head of Department of **Artificial Intelligence and Machine Learning**, for his whole-hearted support in completion of the Entrepreneurial Project.

I am highly indebted to my project guide **Dr. Shyam P Joy [HOD]** for guiding and giving timely advice and suggestions in the successful completion of this project.

My sincere thanks to **Sandhya Kumari[assoc prof]**, mini-project coordinator for having supported the work related to this project.

Last but not the least, I would like to put forward my heartfelt acknowledgement to all my classmates, friends and all those who have directly or indirectly provided their overwhelming support during my seminar work and the development of this report.

TABLE OF CONTENTS

CERTIFICATE.....	2
DECLARATION.....	3
ABSTRACT.....	4
ACKNOWLEDGEMENTS.....	5
1. INTRODUCTION	7
1.1 Overview.....	7
1.2 Problem Statement.....	7
1.3 Objectives	8
1.4 Scope of the Project	8
2. LITERATURE SURVEY.....	9
3. METHODOLOGY.....	10
3.1 SYSTEM DESIGN	10
3.1.1 System Requirements.....	10
3.1.2 System Architecture / High-Level Design (HLD)	11
3.1.3 Database Design (ER Diagram / Schema).....	13
3.1.4 Module Description	13
3.2 IMPLEMENTATION.....	14
3.2.1 Technology Stack.....	14
3.2.2 Implementation of Key Modules	14
4. RESULTS & DISCUSSIONS.....	18
4.1 Testing Methodology	18
4.2 Screenshots of Results	18
4.3 Performance Evaluation.....	21
4.4 Discussion	21
5. CONCLUSION.....	22
5.1 Conclusion	22
5.2 Limitations	22
5.3 Future Scope	22
6. REFERENCES	23
APPENDIX A: SOURCE CODE	24

1. INTRODUCTION

1.1 Overview

In today's digital era, educational institutions are increasingly adopting software solutions to automate administrative and academic processes. Managing faculty workload is one of the most important and challenging tasks in academic departments, especially in engineering colleges where multiple theory subjects, laboratory sessions, mentoring activities, and administrative responsibilities must be assigned efficiently among faculty members. Traditionally, this process is carried out manually using spreadsheets and paper-based records, which often leads to calculation errors, inconsistencies, and increased administrative workload.

With the growing number of courses, faculty members, and academic activities, there is a strong need for an automated system that can simplify workload allocation and generate accurate reports in less time. Faculty workload management is not only important for maintaining balanced teaching responsibilities but also for improving departmental planning, academic scheduling, and resource utilization. Manual allocation methods become difficult to maintain when dealing with large datasets and frequent updates during every academic semester.

The AIML Faculty Load Allocator is developed as a web-based solution to automate the process of faculty workload calculation and report generation. The system processes Excel sheets containing subject allocation data and automatically calculates faculty-wise workload distribution. It supports theory load calculation, laboratory allocation, and TRCPIE workload management while generating structured reports for academic departments. The application is developed using React and Vite for the frontend and FastAPI for the backend, providing a modern, efficient, and user-friendly platform for workload management.

The project demonstrates the practical application of web technologies, backend API development, and data processing techniques in solving real-world academic administration problems. By reducing manual effort and improving accuracy, the system helps departments manage faculty workload more effectively and efficiently.

1.2 Problem Statement

Faculty workload allocation in many educational institutions is still managed manually using spreadsheets and traditional methods. This process is time-consuming and often leads to errors, duplicate entries, and inaccurate workload distribution, especially when handling multiple faculty members and subjects.

Department coordinators spend significant effort preparing and verifying workload reports every semester. As the amount of data increases, maintaining accuracy and consistency becomes difficult. Therefore, there is a need for an automated system that can efficiently process faculty allocation data and generate accurate workload reports with minimal manual effort.

1.3 Objectives

- To develop a web-based application for automating faculty workload allocation.
- To implement automatic calculation of theory, laboratory, and TRCPIE workloads.
- To generate structured and downloadable workload reports for academic departments.
- To reduce manual effort, calculation errors, and report preparation time.
- To provide a simple and user-friendly interface for faculty workload management.
- To improve accuracy and consistency in faculty allocation and report generation.
- To design a system that processes Excel sheets containing faculty and subject allocation data.

1.4 Scope of the Project

The scope of the AIML Faculty Load Allocator is focused on automating faculty workload calculation and report generation within an academic department. The system accepts structured Excel files as input, processes faculty allocation data, calculates workload distribution, and generates organized reports for departmental use. The project includes frontend file upload functionality, backend API processing, Excel data handling, and automated report generation features.

The system is designed mainly for departmental-level faculty workload management and supports workload categories such as theory classes, laboratory sessions, and TRCPIE allocations. It helps administrators and coordinators reduce manual calculations and improve report accuracy.

However, the current version of the project does not include advanced features such as cloud deployment, multi-department integration, role-based authentication, database storage, or real-time synchronization. The project is also limited to processing predefined Excel formats and does not support dynamic timetable generation or institution-wide ERP integration. Future enhancements can extend the system to include these additional functionalities.

2. LITERATURE SURVEY

1. FastAPI Framework

FastAPI is a modern Python framework used for developing APIs and backend applications. It is known for its fast performance, easy syntax, and automatic API documentation features. Many developers use FastAPI for building scalable web applications because it supports asynchronous processing and reduces development time. However, FastAPI itself does not provide any built-in functionality for faculty workload management or Excel-based report generation, which is handled separately in our project.

2. React Framework

React is a JavaScript library used for creating user interfaces for web applications. It follows a component-based approach, making frontend development easier and more organized. React helps in building responsive and interactive applications with better user experience. Existing React applications mainly focus on frontend functionality, but additional backend processing is required for handling workload calculations and automated report generation.

3. Pandas Library

Pandas is a Python library widely used for data analysis and processing. It helps in handling large datasets efficiently and supports operations like filtering, sorting, grouping, and data manipulation. Pandas is very useful for processing Excel sheets and generating structured outputs. However, workload allocation logic and report generation need to be implemented separately based on project requirements.

4. OpenPyXL Library

OpenPyXL is a Python library used for reading and writing Excel files. It allows developers to automate spreadsheet operations such as creating worksheets, formatting cells, and generating reports. Many applications use OpenPyXL for Excel automation, but most of them do not include intelligent faculty workload calculation or a proper web interface. Our project combines Excel automation with a user-friendly web application for easier workload management.

5. Existing Academic Management Systems

Many academic management systems provide features like timetable scheduling, student management, and faculty records. However, most of these systems are large ERP-based applications that are costly and complex to maintain. Smaller departments still depend on manual spreadsheet methods for workload allocation. The proposed AIML Faculty Load Allocator focuses specifically on simplifying faculty workload calculation and generating reports automatically in an easy and efficient way.

3. METHODOLOGY

3.1 SYSTEM DESIGN

3.1.1 System Requirements

The system design explains the structure and working of the AIML Faculty Load Allocator before implementation. The project is designed as a web-based application where users can upload Excel files containing faculty allocation details, and the system automatically processes the data to generate workload reports. The system mainly consists of frontend, backend, and data processing modules that work together to automate workload calculation and report generation.

The frontend is developed using React and Vite, which provides a simple and responsive user interface for file upload and report download. The backend is developed using FastAPI, which handles API requests, file processing, and workload calculation. Pandas and OpenPyXL libraries are used for reading Excel files, processing data, and generating output reports.

The workflow of the system is as follows:

1. User uploads the Excel file through the frontend.
2. The frontend sends the file to the FastAPI backend.
3. The backend validates and processes the uploaded file.
4. Faculty workload calculations are performed.
5. The final report is generated in Excel format.
6. The generated report is made available for download.

Functional Requirements

- The system shall allow users to upload Excel files containing faculty allocation data.
- The system shall process uploaded Excel sheets automatically.
- The system shall calculate theory workload for faculty members.
- The system shall calculate laboratory workload allocation.
- The system shall generate TRCPIE workload reports.
- The system shall generate downloadable Excel reports.
- The system shall validate uploaded file formats and data consistency.
- The user should be able to view generated results through the web interface.

Non-Functional Requirements

- The system should provide fast and efficient processing of Excel files.
- The application should maintain accuracy and consistency in workload calculations.
- The system should be user-friendly and easy to operate.
- The application should support scalability for future enhancements.
- The system should provide reliable report generation without data loss.
- The application should support maintainability and modular development.

Hardware and Software Requirements

Category	Development
OS	Windows 10 / Ubuntu 22.04
Processor	Intel Core i5 or above
GPU	NA
RAM	8 GB RAM
Software	Python 3.x, VS Code, Node.js
Libraries	Pandas, OpenPyXL, NumPy

3.1.2 System Architecture / High-Level Design (HLD)

The AIML Faculty Load Allocator follows a three-layer architecture consisting of the Presentation Layer, Business Logic Layer, and Data Processing Layer. The system is designed to automate faculty workload calculation and report generation using uploaded Excel files.

1. Presentation Layer (Frontend)

The presentation layer is developed using React, Vite, and TypeScript. It provides a simple and interactive web interface where users can upload Excel files and download generated workload reports. This layer manages user interaction and sends requests to the backend server.

- Functions of Presentation Layer:
- File upload interface
- User interaction and navigation
- Display of processing status
- Report download functionality

2. Business Logic Layer (Backend)

The business logic layer is developed using FastAPI. It handles API requests, validates uploaded files, processes faculty workload calculations, and generates reports. This layer contains the main application logic of the system.

Functions of Business Logic Layer:

- API request handling
- File validation
- Faculty workload calculation
- Faculty name matching
- Report generation

3. Data Processing Layer

The data processing layer uses Pandas and OpenPyXL libraries for handling Excel data and report generation. Pandas is used for data manipulation and workload calculation, while OpenPyXL is used for creating structured Excel reports.

Functions of Data Processing Layer:

- Reading Excel sheets
- Data preprocessing
- Workload computation
- Excel report creation

System Workflow

1. The user uploads the Excel file through the frontend interface.
2. The frontend sends the file to the FastAPI backend.
3. The backend validates and processes the uploaded file.
4. Pandas extracts and processes faculty allocation data.
5. Workload calculations are performed for theory, laboratory, and TRCPIE components.
6. OpenPyXL generates the final Excel report.
7. The generated report is returned to the frontend for download.

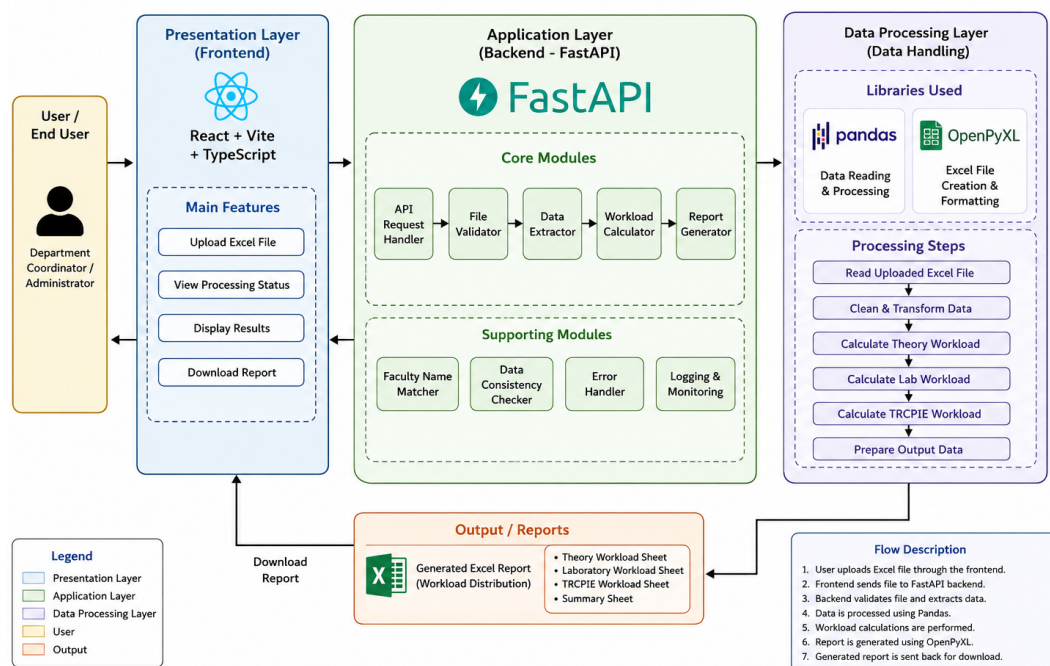


Figure 3.1: System Architecture Diagram

3.1.3 Database Design (ER Diagram / Schema)

The AIML Faculty Load Allocator does not use a traditional database such as MySQL or MongoDB. Instead, the system processes uploaded Excel files directly using Pandas DataFrames. All faculty allocation data is temporarily stored in memory during processing and the final workload reports are generated as downloadable Excel files using the OpenPyXL library.

The uploaded Excel sheet contains details such as faculty names, subject information, workload units, laboratory allocation, and TRCPIE workload data. The backend processes this data, performs workload calculations, and generates structured Excel reports. Since the project is file-based, no permanent database storage is maintained.

The system mainly works with the following logical entities:

- Faculty Details
- Subject Allocation Data
- Workload Calculation Data
- Generated Excel Reports

NA

Figure 3.3: Entity Relationship Diagram

3.1.4 Module Description

No.	Module Name	Description
1	User Authentication	This module allows users to upload Excel files containing faculty allocation details through the frontend interface. It connects with the backend processing module by sending uploaded files to the FastAPI server for validation and processing.
2	Data Input / Pre-processing	This module reads the uploaded Excel sheets using Pandas and performs data preprocessing operations such as validation, cleaning, and formatting. It prepares the dataset for workload calculation and ensures consistency in faculty and subject data.
3	Core Processing / ML Model	This module performs the main workload calculation logic of the system. It calculates theory workload, laboratory allocation, and TRCPIE workload distribution using the processed data and sends the results to the report generation module.
4	Output / Result Display	This module displays processing results and provides downloadable workload reports through the frontend interface. It connects the backend-generated Excel reports with the user interface for easy access and download.
5	Report Generation Module	This module generates structured Excel reports using OpenPyXL after all workload calculations are completed. It organizes faculty allocation details and workload summaries into multiple sheets for final output generation.

3.2 IMPLEMENTATION

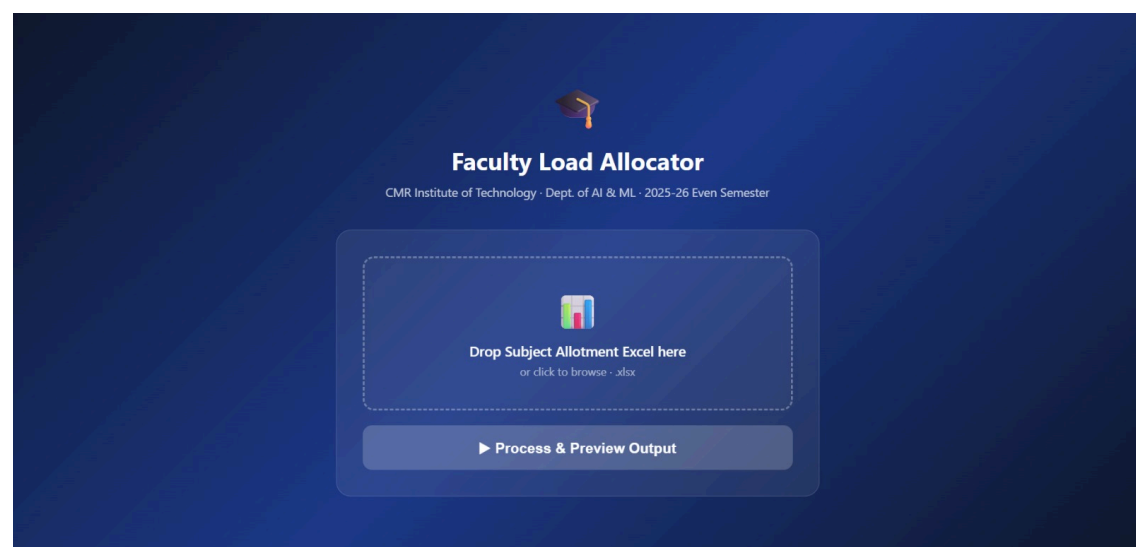
This chapter explains how the AIML Faculty Load Allocator was developed and implemented. The system was built as a web-based application using React and Vite for the frontend and FastAPI for the backend. Pandas and OpenPyXL libraries were used for Excel file processing and report generation. The implementation mainly focused on automating faculty workload calculation and simplifying report preparation.

3.2.1 Technology Stack

Layer	Technology / Tool	Reason for Choice
Frontend	React + Vite + TypeScript	provides fast, responsive, and component-based user interface development
Backend	FastAPI	High-performance Python framework with simple API development
Database	Not Applicable	The project processes Excel files directly without using a database
Data Processing Library	Pandas	Efficient handling and manipulation of Excel datasets
Excel Report Generation	OpenPyXL	Supports Excel file reading, writing, and formatting
Development Environment	VS Code	Lightweight and widely used code editor
Deployment	Localhost / FastAPI Server	Simple deployment and testing during development

3.2.2 Implementation of Key Modules

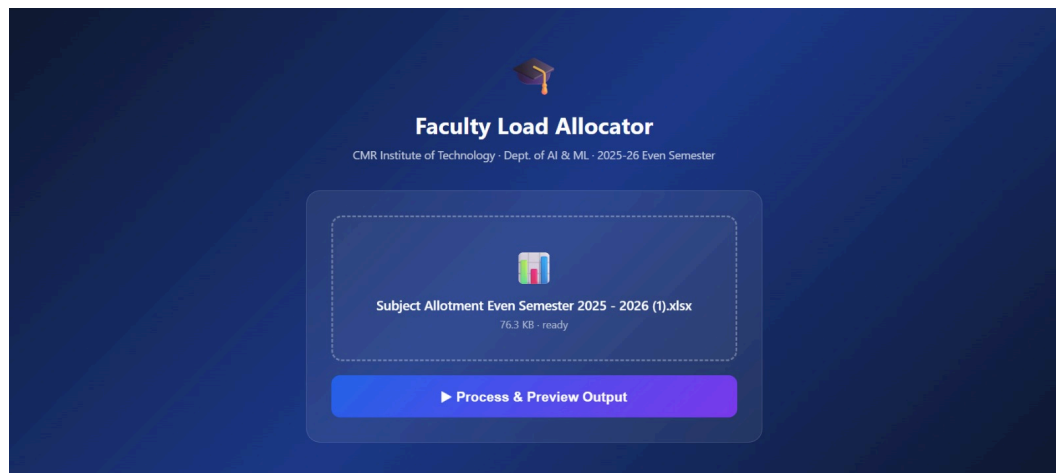
Module 1: [File Upload Module]



The File Upload Module was implemented using React components in the frontend. This module allows users to select and upload Excel files containing faculty allocation details. The uploaded file is sent to the FastAPI backend using HTTP POST requests.

The frontend handles file selection and validation before sending the file to the backend server. The backend API receives the file and performs initial validation checks to ensure proper file format and structure.

Module 2: [Data Input and Pre-processing Module]



This module was implemented using the Pandas library in Python. The uploaded Excel file is read using Pandas DataFrames, and the required faculty allocation data is extracted for processing.

The module performs preprocessing tasks such as checking missing values, validating column names, removing inconsistent entries, and organizing the data into structured format. This processed data is then forwarded to the workload calculation module.

Module 3: [Workload Calculation Module]

AIML Faculty Core Load — 2025-26 Even Semester													
CMR Institute of Technology - AI Faculty - 6000 Total Teaching Units													
Subject Allocation — 24 faculty													
#	Name	Designation	Theory	Lab	Teaching Units	B	C	P	I	E	Admin	Total	
1	Mr. Adarsh Benjamin	Asst Prof	1.5000	1	2.5000	0.3375	0.6	0.3	—	—	—	4	
2	Mr. Navey Jacob (PhD)	Asst Prof	1	1	2	—	—	—	0.8	0.2	1	4	
3	Mr. Prabhakara S	Asst Prof	1	1	2	—	—	—	0.7	1.3	—	4	
4	Mr. Naveya VK (PhD)	Asst Prof	1	1	2	1.1	0.6	0.3	—	—	—	4	
5	Mr. Priyanka ST (PhD)	Asst Prof	1	1	2	—	0.6	0.4	—	—	1	4	
6	Mr. Jeevitha R(PhD)	Asst Prof	1	3	4	—	—	—	—	—	—	4	
7	Mr. Anshika Shukla (PhD)	Asst Prof	2	—	2	1	—	1	—	—	—	4	
8	Mr. Divya Kumar (PhD)	Asst Prof	1	2	3	0.7	—	0.3	—	—	—	4	
9	Mr. Shikhi PT	Asst Prof	1	2	3	—	0.6	0.4	—	—	—	4	
10	Mr. Laya N	Asst Prof	1	2	3	1	—	—	—	—	—	4	
11	Revathi S (PhD)	Asst Prof	1	1	2	1	0.6	0.4	—	—	—	4	
12	Sandhya Kumar (PhD)	Asst Prof	1	1	2	2	—	—	—	—	—	4	
13	Divyanshi LS	Asst Prof	1	2	3	1	—	—	—	—	—	4	
14	Pritya Trivedi	Asst Prof	1	2	3	1	—	—	—	—	—	4	
15	Dr. Shalini H	Asst Prof	1	1	2	1.8	—	0.2	—	—	—	4	
16	S Saanthya	Asst Prof	2	2	4	—	—	—	—	—	—	4	
17	Sneethasudha Purayathaya	Asst Prof	1	3	4	—	—	—	—	—	—	4	
18	Vinoda S	Asst Prof	0.5	3	3.5	0.5	—	—	—	—	—	4	
19	Saurav Kumar Topu	Asst Prof	0.5	2	2.5	1.5	—	—	—	—	—	4	
20	Mertle Dave	Asst Prof	1	3	4	—	—	—	—	—	—	4	
21	Manasola M	Asst Prof	1	1	2	2	—	—	—	—	—	4	
22	Jhathaya Udayagiri	Asst Prof	2	1	3	1	—	—	—	—	—	4	
23	Mr. Batta Saranya	Asst Prof	0.5	3	3.5	0.5	—	—	—	—	—	4	

The Workload Calculation Module is the core processing unit of the project. This module calculates theory workload, laboratory workload, and TRCPIE allocation based on the uploaded dataset.

Different workload rules and calculations were implemented using Python functions and DataFrame operations. The module also performs faculty name matching to avoid duplicate entries and improve consistency in report generation.

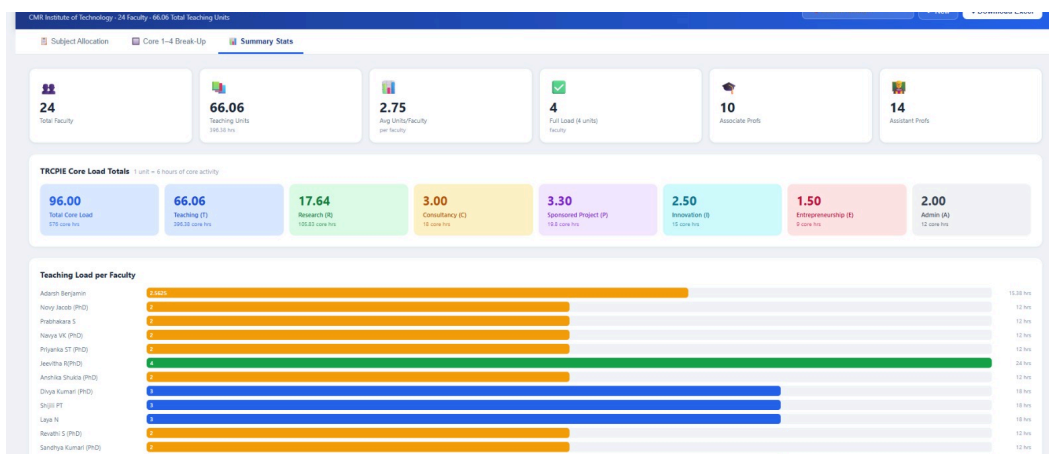
Module 4:[Output and Result Display Module]

S.No	Name of the Faculty	Designation	Theory	Lab	Units for TRCPIE					Admin	D Load	Total load	Extra Load
					Teaching	Research	Consultancy	Project	Innovation	Entrepreneurship	Admin	D Load	Extra Load
1	Mr. Adarsh	Assistant Prof	1.5625	1	2.5625	0.5375	0.6	0.3				4	0
2	Ms. Novy	Associate Prof	1	1	2				0.8	0.2	1	4	0
3	Mr. Prabhu	Assistant Prof	1	1	2				0.7	1.3		4	0
4	Ms. Navya	Associate Prof	1	1	2	1.1	0.6	0.3				4	0
5	Ms. Priya	Associate Prof	1	1	2		0.6	0.4			1	4	0
6	Ms. Jeevit	Associate Prof	1	3	4							4	0
7	Ms. Anshul	Associate Prof	2		2	1		1				4	0
8	Ms. Divya	Associate Prof	1	2	3	0.7		0.3				4	0
9	Ms. Shijili	Assistant Prof	1	2	3		0.6	0.4				4	0
10	Ms. Laya	Assistant Prof	1	2	3	1						4	0
11	Revathi S	Associate Prof	1	1	2	1	0.6	0.4				4	0
12	Sandhya K	Associate Prof	1	1	2	2						4	0
13	Divyaman	Assistant Prof	1	2	3	1						4	0
14	Priya Trive	Assistant Prof	1	2	3	1						4	0
15	Dr. Shalme	Associate Prof	1	1	2	1.8		0.2				4	0
16	S Santhiya	Assistant Prof	2	2	4							4	0
17	Swasthisu	Assistant Prof	1	3	4							4	0
18	Vinuta S	Assistant Prof	0.5	3	3.5	0.5						4	0
19	Saurav Kur	Assistant Prof	0.5	2	2.5	1.5						4	0
20	Meeta Dav	Assistant Prof	1	3	4							4	0
21	Manimala	Assistant Prof	1	1	2	2						4	0

This module connects the backend-generated output with the frontend interface. After workload processing is completed, the generated Excel report is returned to the frontend for download.

The frontend displays processing status and allows users to download the final report easily. This module improves user interaction and simplifies the overall workflow of the application.

Module 5: [Report Generation Module]



The Report Generation Module was implemented using the OpenPyXL library. This module generates structured Excel reports containing faculty workload summaries and allocation details.

Multiple worksheets are created in the output Excel file to organize theory load, laboratory load, and TRCPIE allocation separately. The generated reports are properly formatted and made available for download through the frontend interface.

4. RESULTS & DISCUSSIONS

The AIML Faculty Load Allocator was tested using different Excel datasets to verify the correctness of workload calculation, report generation, and overall system functionality. The testing process was carried out to ensure that all modules worked correctly both individually and as a complete integrated system. The application successfully processed uploaded Excel files and generated structured workload reports without major errors. The generated reports included theory workload, laboratory workload, and TRCPIE allocation details with proper formatting and accurate calculations.

The frontend and backend modules communicated successfully through API integration, allowing smooth file upload, processing, and report download operations. The system reduced manual effort and improved the accuracy of faculty workload calculations compared to traditional spreadsheet-based methods.

4.1 Testing Methodology

The system was tested using Unit Testing and Integration Testing methods. Unit testing was performed on individual modules such as file upload, Excel processing, workload calculation, and report generation to verify their functionality independently. Integration testing was conducted to ensure proper communication between the frontend, backend, and data processing modules.

Black-box testing was also performed by providing different Excel files as input and verifying whether the generated reports matched the expected output. Performance testing was carried out with multiple datasets to check processing speed and report generation efficiency. The testing process confirmed that the system was reliable, accurate, and capable of handling workload allocation operations efficiently.

4.2 Screenshots of Results

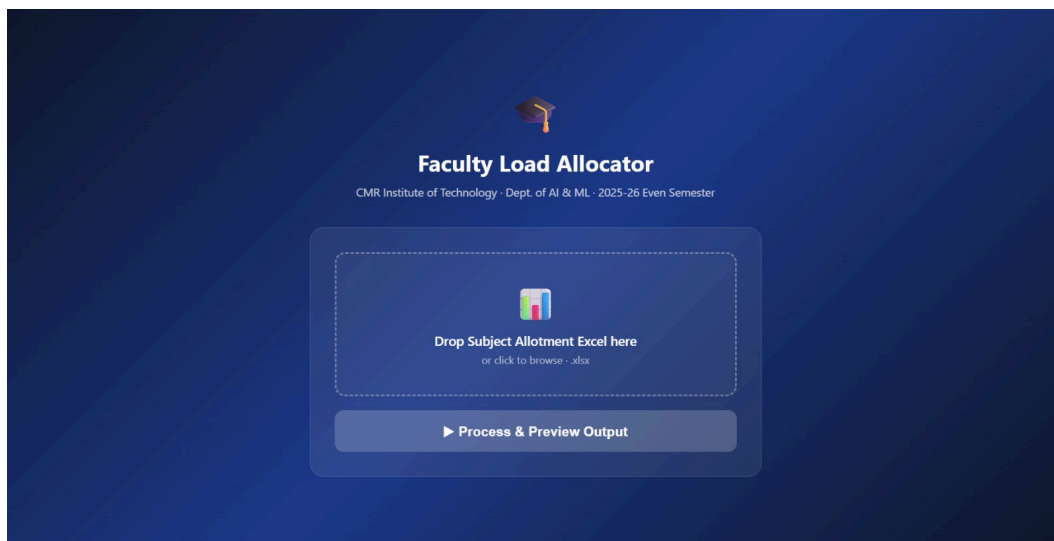


Figure 4.1: [The frontend interface allows users to upload Excel files containing faculty allocation details. The page provides a simple and user-friendly design for selecting and submitting files for processing.]

AIML Faculty Core Load — 2025-26 Even Semester				
Institute of Technology - 24 Faculty - 66.06 Total Teaching Units				
Subject Allocation Core 1-4 Break-Up Summary Stats				
Core 1-4 Break-Up — 24 faculty				
Name	Core 1	Core 2	Core 3	Core 4
Mr. Adarsh Benjamin	TR 23 AIML 4A-BUHK408-Universal Human Values course 0.1875R	TR 23 AIML 4B-BUHK408-Universal Human Values course 0.1875R	TR 23 CSE(AIML) 4A-BUHK408-Universal human values course 0.1875R	TR 23 CSE(AIML) 4A-BCSL404-Analysis & Design of Algorithms Lab (3 Slots)
Ms. Novy Jacob (PhD)	TT 22 AIML 6B-BAI601-Natural Language Processing	TT 22 AIML 6B-BAI601-Natural Language Processing Lab (3 Slots)	IE 0.8 I 0.2 E	TA Admin
Mr. Prabhakara S	TT 23 AIML 4A-BCSL404-Analysis & Design of Algorithms Lab (3 Slots)	TT Semester 1/2-Introduction to AI & Applications	E 1.3 E	I 0.7 I
Ms. Navya VK (PhD)	TT 22 CSE(AIML) 6-BCSL606-Machine Learning -I	TT 22 CSE(AIML) 6-BCSL606-Machine Learning lab (3 Slots)	R 1.1 R	CP 0.6 C 0.3 P
Ms. Priyanka ST (PhD)	TT 22 AIML 6B-BAI602-Machine Learning -I	TT 22 AIML 6B-BAI606-Machine Learning lab (3 Slots)	CP 0.6 C 0.4 P	TA Admin
Ms. Jeevitha R(PhD)	TT 23 AIML 4A-BDSL456C-Ability Enhancement Course (MERN*) (3 Slots)	TT 23 AIML 4B-BCSL403-Database Management Systems	TT 23 AIML 4B-BDSL456C-Ability Enhancement Course (MERN*) (3 Slots)	TT 23 CSE(AIML) 4A-BCSL403-Database Management Systems Lab (3 Slots)
Ms. Anshika Shukla (PhD)	TT 23 CSE(AIML) 4A-TYL P3 Python	TT 22 CSE(AIML) 6-BCI613x-Cloud computing security*	R 1.0 R	P 1.0 P
Ms. Divya Kumari (PhD)	TT 23 AIML 4A-TYL P3 Python	TT 22 AIML 6A-BAI606-Machine Learning lab (3 Slots)	TT 22 AIML 6B-BAI606-Machine Learning lab (3 Slots)	RP 0.7 R 0.3 P
Ms. Shijili PT	TT 23 AIML 4B-BCSL401-Analysis & Design of Algorithms	TT 23 AIML 4B-BCSL404-Analysis & Design of Algorithms Lab (3 Slots)	TT 22 CSE(AIML) 6-BCSL606-Machine Learning lab (3 Slots)	CP 0.6 C 0.4 P
Ms. Laya N	TT 23 AIML 4B-TYL P3 Python	TT 22 AIML 6A-BAI637x-Ability Enhancement Course/Skill Development(DevOps) (3 Slots)	TT 22 AIML 6B-BAI637x-Ability Enhancement Course/Skill Development(DevOps) (3 Slots)	R 1.0 R
Revathi S (PhD)	TT 22 AIML 6A-BAI602-Machine Learning -I	TT 22 AIML 6A-BAI606-Machine Learning lab (3 Slots)	R 1.0 R	CP 0.6 C 0.4 P

AIML Faculty Core Load — 2025-26 Even Semester											
CMR Institute of Technology - 24 Faculty - 66.06 Total Teaching Units											
Subject Allocation Core 1-4 Break-Up Summary Stats											
Subject Allocation — 24 faculty											
#	Name	Designation	Theory	Lab	Teaching Units	R	C	P	I	E	Admin
1	Mr. Adarsh Benjamin	Asst Prof	1.5625	1	2.5625	0.5375	0.6	0.3	—	—	—
2	Ms. Novy Jacob (PhD)	Asst Prof	1	1	2	—	—	—	0.8	0.2	1
3	Mr. Prabhakara S	Asst Prof	1	1	2	—	—	—	0.7	1.3	—
4	Ms. Navya VK (PhD)	Asst Prof	1	1	2	1.1	0.6	0.3	—	—	—
5	Ms. Priyanka ST (PhD)	Asst Prof	1	1	2	—	0.6	0.4	—	—	1
6	Ms. Jeevitha R(PhD)	Asst Prof	1	3	4	—	—	—	—	—	—
7	Ms. Anshika Shukla (PhD)	Asst Prof	2	—	2	1	—	1	—	—	—
8	Ms. Divya Kumari (PhD)	Asst Prof	1	2	3	0.7	—	0.3	—	—	—
9	Ms. Shijili PT	Asst Prof	1	2	3	—	0.6	0.4	—	—	—
10	Ms. Laya N	Asst Prof	1	2	3	1	—	—	—	—	—
11	Revathi S (PhD)	Asst Prof	1	1	2	1	0.6	0.4	—	—	—
12	Sandhya Kumari (PhD)	Asst Prof	1	1	2	2	—	—	—	—	—
13	Divyamani LS	Asst Prof	1	2	3	1	—	—	—	—	—
14	Priya Trivedi	Asst Prof	1	2	3	1	—	—	—	—	—
15	Dr. Shalma H	Asst Prof	1	1	2	1.8	—	0.2	—	—	—
16	S Santhiya	Asst Prof	2	2	4	—	—	—	—	—	—
17	Swasthisudha Puriyathaya	Asst Prof	1	3	4	—	—	—	—	—	—
18	Vinuta S	Asst Prof	0.5	3	3.5	0.5	—	—	—	—	—
19	Saurav Kumar Topo	Asst Prof	0.5	2	2.5	1.5	—	—	—	—	—
20	Meeta Dave	Asst Prof	1	3	4	—	—	—	—	—	—
21	Manimala M	Asst Prof	1	1	2	2	—	—	—	—	—
22	Jhashuva Udayagiri	Asst Prof	2	1	3	1	—	—	—	—	—
23	Ms. Batta Saranya	Asst Prof	0.5	3	3.5	0.5	—	—	—	—	—

Figure 4.2: [After successful processing, the system generates workload allocation details including theory workload, laboratory workload, and TRCPIE calculations. The generated report is displayed in structured format for easy understanding.]

Department of Artificial Intelligence & Machine Learning Academic Year 2025-26 (Even Semester) - Subject and TRCPIE Allotment													
S.No	Name of the Faculty	Designation	Theory	Lab	Units for TRCPIE					Admin	D Load	Total load	Extra Load
					Teaching U	Research U	Consultan	Project Un	Innovation	Entrepre	Admin	D Load	Extra Load
1	Mr. Adarsh	Assistant F	1.5625	1	2.5625	0.5375	0.6	0.3				4	0
2	Ms. Novy J.	Associate F	1	1	2				0.8	0.2	1	4	0
3	Mr. Prabhakar	Assistant F	1	1	2				0.7	1.3		4	0
4	Ms. Navya	Associate F	1	1	2	1.1	0.6	0.3				4	0
5	Ms. Priyanka	Associate F	1	1	2		0.6	0.4			1	4	0
6	Ms. Jeevit	Associate F	1	3	4							4	0
7	Ms. Anshul	Associate F	2		2	1		1				4	0
8	Ms. Divya	Associate F	1	2	3	0.7		0.3				4	0
9	Ms. Shijili	Assistant F	1	2	3		0.6	0.4				4	0
10	Ms. Laya M	Assistant F	1	2	3	1						4	0
11	Revathi S (	Associate F	1	1	2	1	0.6	0.4				4	0
12	Sandhya K	Associate F	1	1	2	2						4	0
13	Divyamani	Assistant F	1	2	3	1						4	0
14	Priya Trive	Assistant F	1	2	3	1						4	0
15	Dr. Shalme	Associate F	1	1	2	1.8		0.2				4	0
16	S Santhiya	Assistant F	2	2	4							4	0
17	Swasthisur	Assistant F	1	3	4							4	0
18	Vinuta S	Assistant F	0.5	3	3.5	0.5						4	0
19	Saurav Kur	Assistant F	0.5	2	2.5	1.5						4	0
20	Meeta Dav	Assistant F	1	3	4							4	0
21	Manimala	Assistant F	1	1	2	2						4	0

Figures 4.3.[1,2,3]: [The final output is generated as a downloadable Excel file containing multiple sheets for workload distribution and faculty allocation summaries.]

4.3 Performance Evaluation

The AIML Faculty Load Allocator does not use a Machine Learning model, so metrics such as Accuracy, Precision, Recall, and F1-Score are not applicable. The system performance was evaluated based on processing speed, workload calculation accuracy, and report generation efficiency.

The application was tested using multiple Excel datasets, and the system successfully processed uploaded files and generated workload reports without major errors. Pandas-based data processing improved efficiency compared to manual spreadsheet calculations.

Parameter

- File Upload Response
- Processing Speed
- Workload Calculation Accuracy
- Report Generation
- Frontend-Backend Communication

Observation

Successful for valid Excel files
Fast processing of datasets
Accurate workload generation
Excel reports generated successfully
Smooth API response and file transfer

4.4 Discussion

The AIML Faculty Load Allocator successfully achieved the main objective of automating faculty workload calculation and report generation. The system reduced the dependency on manual spreadsheet calculations and minimized errors caused by repetitive data entry and inconsistent faculty naming formats.

The integration of React frontend and FastAPI backend provided smooth communication between the user interface and processing modules. Pandas and OpenPyXL libraries simplified Excel data processing and report generation. The generated reports were accurate, properly formatted, and easy to understand.

One of the major achievements of the project was the reduction in workload preparation time compared to manual methods. The system was able to process uploaded Excel files efficiently and generate workload summaries automatically. The project also provided practical exposure to frontend development, backend API integration, Excel automation, and real-world problem solving.

Overall, the project demonstrated how web technologies and Python-based data processing can be used effectively to solve academic administration problems and improve efficiency in faculty workload management.

5. CONCLUSION

5.1 Conclusion

The AIML Faculty Load Allocator was developed to solve the problem of manual faculty workload calculation and report generation in academic departments. In many institutions, workload allocation is still managed using spreadsheets and manual calculations, which consume a lot of time and may lead to errors and inconsistencies. To overcome these problems, a web-based system was developed that automates the process of workload calculation using uploaded Excel sheets.

The project successfully achieved its objectives by implementing automated theory load, laboratory load, and TRCPIE workload calculation along with Excel report generation. The application was developed using React and Vite for the frontend and FastAPI for the backend, while Pandas and OpenPyXL were used for data processing and Excel automation. The system improved accuracy, reduced manual effort, and simplified the overall workload management process.

Through this project, practical knowledge was gained in frontend development, backend API integration, Excel file processing, and web application development. The project also helped in understanding real-world problem solving, teamwork, and software development methodologies.

5.2 Limitations

- The system currently supports only predefined Excel file formats.
- The project does not include user authentication or role-based access control.
- The application is limited to department-level workload management.
- Cloud deployment and real-time database integration are not implemented.
- The system does not support automatic timetable generation.

5.3 Future Scope

- Integration with college ERP systems for centralized academic management.
- Implementation of user login and role-based authentication features.
- Cloud deployment for remote access and better scalability.
- Addition of graphical dashboards and workload analytics.
- Support for multiple departments and institution-wide workload management.
- Integration of automatic timetable generation and workload balancing features.

6. REFERENCES

- [1] Sebastián Ramírez, FastAPI Documentation, FastAPI Official Documentation, Available: <https://fastapi.tiangolo.com/>
- [2] Meta Platforms Inc., React Documentation, React Official Documentation, Available: <https://react.dev/>
- [3] Wes McKinney, Python for Data Analysis, 3rd Edition, O'Reilly Media, 2022.
- [4] The Pandas Development Team, Pandas Documentation, Available: <https://pandas.pydata.org/docs/>
- [5] Charlie Clark, OpenPyXL Documentation, Available: <https://openpyxl.readthedocs.io/>
- [6] Microsoft Corporation, TypeScript Documentation, Available: <https://www.typescriptlang.org/docs/>
- [7] Evan You, Vite Documentation, Available: <https://vitejs.dev/guide/>
- [8] Ian Sommerville, Software Engineering, 10th Edition, Pearson Education, 2015.
- [9] Roger S. Pressman and Bruce R. Maxim, Software Engineering: A Practitioner's Approach, 8th Edition, McGraw-Hill Education, 2014.
- [10] Al Sweigart, Automate the Boring Stuff with Python, 2nd Edition, No Starch Press, 2019.

APPENDIX A: SOURCE CODE

The complete source code of the AIML Faculty Load Allocator project is available in the GitHub repository mentioned below. The repository contains frontend, backend, Excel processing, and report generation modules used in the project.

GitHub Repository: <https://github.com/shwetapawar272006/>

[AIML_faculty_load_allocator](https://github.com/shwetapawar272006/AIML_faculty_load_allocator)

A.1 Backend Main File – backend/main.py

GitHub File Link:

https://github.com/shwetapawar272006/AIML_faculty_load_allocator/blob/main/backend/main.py

A.2 Excel Processing Module – backend/processor.py

GitHub File Link:

https://github.com/shwetapawar272006/AIML_faculty_load_allocator/blob/main/backend/processor.py

A.3 Frontend Main File – frontend/src/App.tsx

GitHub File Link:

https://github.com/shwetapawar272006/AIML_faculty_load_allocator/blob/main/frontend/src/App.tsx

A.4 File Upload Component – frontend/src/components/FileUpload.tsx

GitHub File Link:

https://github.com/shwetapawar272006/AIML_faculty_load_allocator/blob/main/frontend/src/components/FileUpload.tsx

